CSE470: Software Engineering

Software Metrics Part 2

Software measurement and metrics

- Software measurement is concerned with deriving a numeric value for an attribute of a software product or process.
- This allows for objective comparisons between techniques and processes.
- Although some companies have introduced measurement programmes, most organisations still don't make systematic use of software measurement.
- There are few established standards in this area

Fan-in/Fan-out, Length of code

Fan-in/Fan-out

- Fan-in is a measure of the number of functions or methods that call another function or method (say X). Fan-out is the number of functions that are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of X may be high because of the complexity of the control logic needed to coordinate the called components.
- Read: Medium article

Length of code

- This is a measure of the size of a program. Generally, the larger the size of the code of a component, the more complex and error-prone that component is likely to be. Length of code has been shown to be one of the most reliable metrics for predicting error-proneness in components

Specialization Index (SIX)

The Specialization Index metric measures the extent to which subclasses override their ancestors classes.

This index is the ratio between the number of overridden methods and total number of methods in a Class, weighted by the depth of inheritance for this class.

The metric provides a percentage, where the class contains at least one operation. For a root class, the specialization indicator is 0. Nominal range is between 0 % and 120 %.

Specialization Index (SIX)

$$SIX = \frac{NMO \times DIT}{NMO+NMA+NMI} * 100\%$$

Where,
DIT = Depth of Inheritance
NMO = Number of Overridden Methods
NMA = Number of Methods added
NMI = Number of Inherited methods

Example - Question

```
class Person:
    def read(self):
        pass
    def display(self):
        pass
```

```
class Student(Person):
    def read(self):
        pass
    def display(self):
        pass
    def getAvg(self):
        pass
```

```
class GradStudent(Student):
    def read(self):
        pass
    def display(self):
        pass
    def workStatus(self):
        pass
```

Calculate the SIX value for **GradStudent** Class

Example - Solution

For GradStudent Class,

DIT = 2 (GradStudent -> Student -> Person)
NMI = 1 (getAvg())
NMA = 1 (workStatus())
NMO = 2 (read(), display())

SIX = (2*2)/(1+1+2) = 1 * 100% = 100%

Defect Removal Efficiency (DRE)

- A defect is found when the application does not conform to the requirement specification.
- A mistake in coding is called Error
- An average DRE score is usually around 85% across a full testing program.
- DRE = E / (E + D) where:
  - E is the number of errors found before delivery of the software to the end-user
  - D is the number of defects found after delivery.
- We found 100 defects during the testing phase and then later, say within 90 days after software release (in production), found 5 defects,
- DRE = 100/(100+5) = 95.2%